



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
Disciplina: ALGORITMO E ESTRUTURA DE DADOS I

Semestre: 2026.2

Data: ____/____/____
PROFESSORA: Rosana Rego

— Exercício 6: Unidade I —

Manipulação de arquivos: texto e binário

Nome: _____

Nota: _____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade6.c`, evolução da `atividade5.c`, cujo objetivo é demonstrar, de forma visual e interativa (usando a biblioteca **raylib**), os seguintes conceitos de linguagem C:

- Abertura e fechamento de arquivos com `fopen/fclose`, incluindo o tratamento do erro quando `fopen` retorna `NULL`;
- Diferença entre **modo texto** ("**a**", "**r**") e **modo binário** ("**wb**", "**rb**");
- Escrita/leitura de dados legíveis por humanos com `fprintf/fscanf` (histórico de pontuação);
- Escrita/leitura do estado completo do jogo, byte a byte, com `fwrite/fread` (save/load binário);
- Revisão: ponteiros, alocação dinâmica, *struct*, **enum**, **union** e vetor de ponteiros para *struct* (*Entidade*), reaproveitados da atividade 5.

O programa é o mesmo sistema de entidades das atividades anteriores, agora com três novas teclas: F5 salva a pontuação em um arquivo de texto, F6 salva o jogo inteiro em um arquivo binário, e F9 recarrega esse jogo salvo.

1 Arquivo de texto: histórico de pontuação

A cada vez que o jogador pressiona F5, a pontuação atual é anexada ao final do arquivo `placar.txt`, em modo texto ("**a**" de *append*). Como o arquivo é texto puro, ele pode ser aberto e lido em qualquer editor:

```
1 void salvarPlacarTexto(int pontuacao) {
2     FILE *arquivo = fopen(ARQUIVO_PLACAR, "a"); // "a": anexa ao
3         final, modo texto
4     if (arquivo == NULL) return;
5
6     fprintf(arquivo, "%d\n", pontuacao);
7     fclose(arquivo);
8 }
```

Para descobrir o recorde já alcançado, o programa lê o arquivo inteiro em modo "**r**" e percorre os valores com `fscanf`, guardando o maior deles:

```

1 int lerMelhorPontuacao(void) {
2     FILE *arquivo = fopen(ARQUIVO_PLACAR, "r"); // "r": leitura, modo
           texto
3     if (arquivo == NULL) return 0;
4
5     int melhor = 0, valor = 0;
6     while (fscanf(arquivo, "%d", &valor) == 1) {
7         if (valor > melhor) melhor = valor;
8     }
9     fclose(arquivo);
10    return melhor;
11 }

```

2 Arquivo binário: salvando o jogo inteiro

Diferentemente do placar, o **estado completo do jogo** (todas as entidades) não precisa ser legível por humanos – por isso é gravado em modo **binário** ("wb"), com `fwrite` copiando os *bytes* de cada *struct* diretamente para o arquivo:

```

1 bool salvarJogoBinario(void) {
2     FILE *arquivo = fopen(ARQUIVO_SAVE, "wb"); // "wb": escrita, modo
           binario
3     if (arquivo == NULL) return false;
4
5     fwrite(&totalEntidades, sizeof(int), 1, arquivo);
6     for (int i = 0; i < totalEntidades; i++) {
7         fwrite(vetorEntidades[i], sizeof(Entidade), 1, arquivo); //
           grava a struct inteira, byte a byte
8     }
9
10    fclose(arquivo);
11    return true;
12 }

```

Note que primeiro é gravado `totalEntidades` (um único `int`) e só depois cada `Entidade`: ao carregar o arquivo, o programa precisa saber antecipadamente quantas *structs* deve ler.

3 Arquivo binário: carregando o jogo

Para carregar, o processo é o inverso: primeiro lê-se o total gravado, depois aloca-se (com `malloc`) uma nova `Entidade` para cada registro lido com `fread`:

```

1 bool carregarJogoBinario(void) {
2     FILE *arquivo = fopen(ARQUIVO_SAVE, "rb"); // "rb": leitura, modo
           binario
3     if (arquivo == NULL) return false;
4
5     int totalSalvo = 0;
6     if (fread(&totalSalvo, sizeof(int), 1, arquivo) != 1) {
7         fclose(arquivo);
8         return false;
9     }
10
11    liberarTodasEntidades();
12    for (int i = 0; i < totalSalvo; i++) {
13        Entidade *e = (Entidade *)malloc(sizeof(Entidade));

```

```

14         if (fread(e, sizeof(Entidade), 1, arquivo) != 1) {
15             free(e);
16             break;
17         }
18         adicionarEntidade(e);
19     }
20
21     fclose(arquivo);
22     return true;
23 }

```

Como as entidades antigas são liberadas e novas são alocadas em endereços diferentes, o ponteiro `jogador` do `main` precisa ser atualizado após o carregamento: `jogador = vetorEntidades[0];`.

Ciclo de vida dos arquivos

Etapa	Texto (placar.txt)	Binário (save.bin)
Abertura	<code>fopen(..., "a") / fopen(..., "r")</code>	<code>fopen(..., "wb") / fopen(..., "rb")</code>
Escrita	<code>fprintf(arquivo, "%d\n", ...)</code>	<code>fwrite(ponteiro, sizeof(...), 1, arquivo)</code>
Leitura	<code>fscanf(arquivo, "%d", ...)</code>	<code>fread(ponteiro, sizeof(...), 1, arquivo)</code>
Fechamento	<code>fclose(arquivo)</code>	<code>fclose(arquivo)</code>

Assim como `malloc` exige um `free` correspondente, todo `fopen` exige um `fclose` – caso contrário, os dados podem não ser gravados no disco (ficam apenas em um *buffer* interno).

4 Exercícios

Exercício 1 — Salvando o nome do jogador

Atualmente `placar.txt` guarda apenas o número da pontuação, uma por linha.

Tarefa: peça o nome do jogador no início do programa (uma string curta) e grave, em cada linha do placar, o nome seguido da pontuação.

Dicas:

- Declare `char nomeJogador[16]` no `main` e use `GetCharPressed()` ou um valor fixo para simplificar.
- Altere `fprintf` para o formato `"%s %d\textbackslash n"`, passando `nomeJogador` e `pontuacao`.
- Ao ler, use `fscanf(arquivo, "%15s %d", nomeLido, &valor)` para ler par a par.

Exercício 2 — Apagando o jogo salvo

Atualmente não existe forma de descartar um `save.bin` antigo.

Tarefa: associe a tecla `KEY_DELETE` para apagar o arquivo `save.bin`, exibindo uma mensagem de confirmação na tela.

Dicas:

- Utilize a função `remove("save.bin")` da biblioteca `<stdio.h>`, que devolve 0 em caso de sucesso.
- Se o arquivo não existir, `remove` retorna um valor diferente de zero: trate esse caso com uma mensagem diferente ("Nenhum save encontrado").
- Reaproveite a variável `mensagem/tempoMensagem` já existente para mostrar o resultado por alguns segundos.